Laboratory 1: Simulink Demo

1 Introduction

The goal of Laboratory 1 is to introduce and familiarize students with Simulink. Simulink is a software distributed with MATLAB that uses a graphical interface to implement simulations of dynamic systems, implemented as block diagrams. Simulink is fully integrated with MATLAB's workspace, which enables seamless data transfer between the two softwares. The Simulink Onramp method will be used to introduce basic blocks and functionalities and then the closed-loop control of a train.

When it comes to the simulation of large cyber-physical systems (industrial plants including control system, robots, smart-grids), Simulink has limitations when it comes to the number of equations and variables and the fact that the physical equations are modeled using causal equations (clear flow of information from input to output). However, it is this limitation that makes Simulink compelling in automatic control: it is easy to implement the block-diagrams that you learn in the theory. Moreover, the learning curve of Simulink is not steep.

2 Simulink Onramp

Simulink Onramp is an online tutorial created by MathWorks to introduce new users to Simulink. Students at LTU have full access to these tutorials and we recommend that you start with them if you have never used Simulink before. Use the following link:

`https://matlabacademy.mathworks.com/details/simulink-onramp/simulink`

3 Example: Train model

Now that you are familiar with basic operations in Simulink, you can start to use it to implement a model of a train. This example is extracted from the website `https://ctms.engin.umich.edu/CTMS/index.php?example=Introduction&section=SimulinkModeling`, which has other interesting processes if you are interested in more practice. The train model consists of two parts: the engine and a car. It is assumed that the train only moves in one direction (along the tracks) and your goal is to design a control system to regulate the train's speed. The train is modeled as two masses M_1 and M_2, connected by a spring with elasticity constant k. The engine produces a force F on M_1 and the wheels have a friction constant μ.

3.1 Free body diagram

In order to model a system in Simulink, it is necessary to know the models' equations. Those, for physical systems, are usually based on a free body diagram, where the masses are separated and all forces acting on them are depicted. Assuming that the forces in the vertical plan cancel each other, we can focus on the horizontal axis. According to the Newton's law:

$$\Sigma F_1 = M_1 \ddot{x}_1 \tag{1}$$

$$\Sigma F_2 = M_1 \ddot{x}_2 \tag{2}$$

Where $\ddot{x}_i$ is the acceleration of mass i. Figure 1 summarizes which forces acts on the masses.

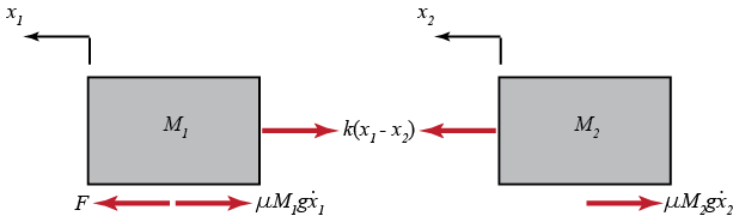

Figure 1: Figure extracted from [1].

3.2 Simulink model

Once the dynamic equations that define how the train's states (position, velocity and acceleration) evolve over time are available from interpreting the free body diagram, you can start implementing the model (set of dynamic equations) in Simulink. Remember that the lines in Simulink represent signals (states) and the blocks are mathematical or logical operations performed on the signals.

Open a new Simulink model, add two Sum blocks (in Math operations) to your model. Name the summations blocks to Sum_F1 and Sum_F2. The output of these blocks will represent the sum of forces acting om each mass. In order to isolate the acceleration, multiply the output of the summation blocks by $\frac{1}{M_1}$ and $\frac{1}{M_2}$, respectively. This is done by dragging Gain block (in Math Operations) twice. Doubble-click on each Gain block and name them as $1/M1$ and $1/M2$. Connect (draw a line between) the output of the Sum block to the input of the Gain block. Observe that if the name of the block is too long for its icon, only a 'K' will show on screen. Resize the blocks to read the full text.

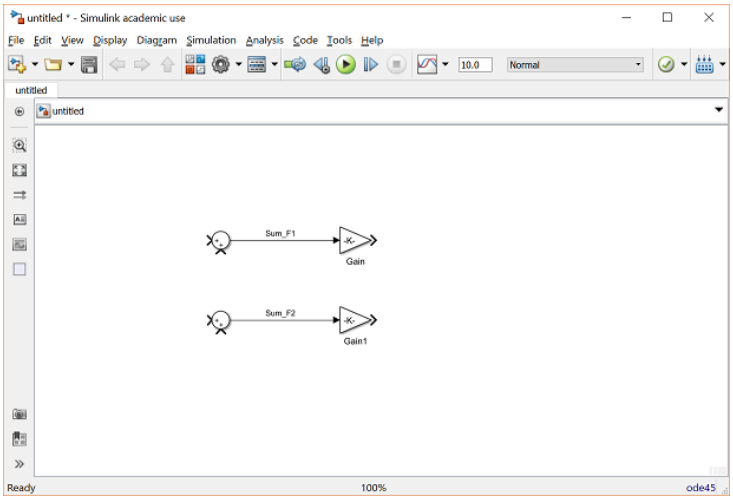

Figure 2: Sum and Gain blocks, extracted from [1].

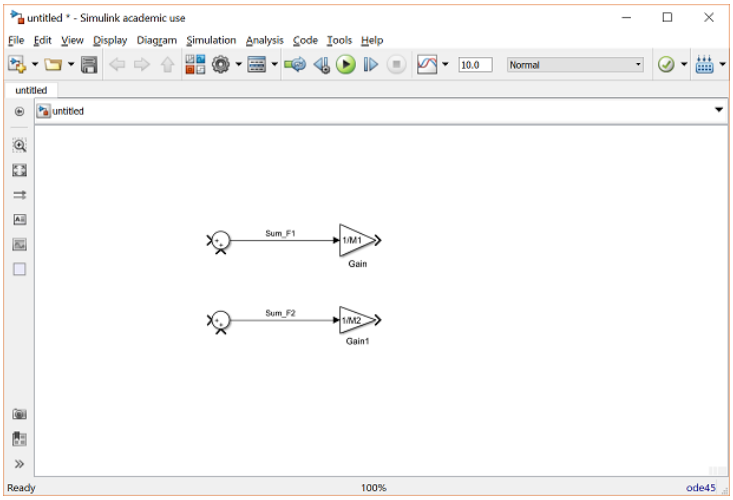

Figure 3: Resized sum and Gain blocks, extracted from [1].

Now we have a signal (line) that is equivalent to the acceleration of the masses. Since we are also interested in knowing how their velocity and position vary over time, we need to integrate the acceleration signal twice. This is done by the Integrator $1/s$ block (in Continuous). Add two Integrator blocks, for each mass and connect the blocks according to the figure below.

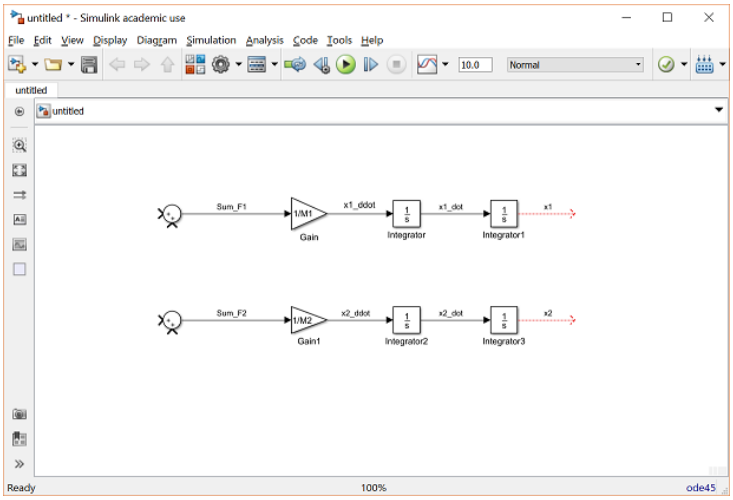

Figure 4: From acceleration to velocity to position, extracted from [1].

Now we have all the needed signals calculated by Simulink, but there is no way to visualize their values during a simulation. To fix that, add Scope blocks (in Sinks) for the positions x_1 and x_2. Not only blocks, but even signals can be named in Simulink. Double-click on the signals x_1 and x_2 and name them accordingly. This will make it easier to analyze the Scope object later on, as the signals will be identified according to their names. If you compare the dynamic equations from the free body diagram and the equations implemented in Simulink, you should note that there are some forces missing in the Sum block. In order to fix that, first change the number of input signals in Sum to M_1 by double-clicking the Sum block and changing the field 'List of signs' to $|+--$.

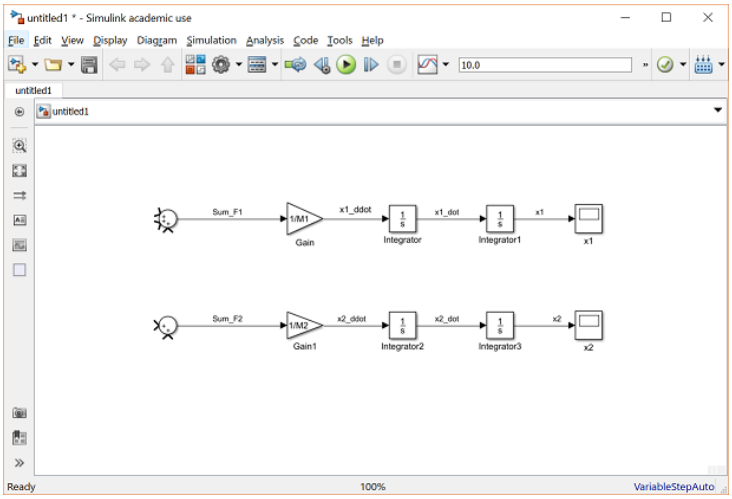

Figure 5: Adding new input to Sum, extracted from [1].

The first force acting in M_1 is the force generated by the engine, F. This is considered in this example as an external force, and thus defined by a Signal Generator block (in Sources). Connect the output of Signal Generator to Sum_1 and name this signal as F. Next, the friction force is a function of the mass' velocity: $F_{friction} = \mu * M_1 * v_1$. In order to implement this in Simulink, connect the velocity signal (output of the first Integrator block) to a Gain block $mu * g * M1$ and then connect the output of the Gain block to the Sum_ 1 block.

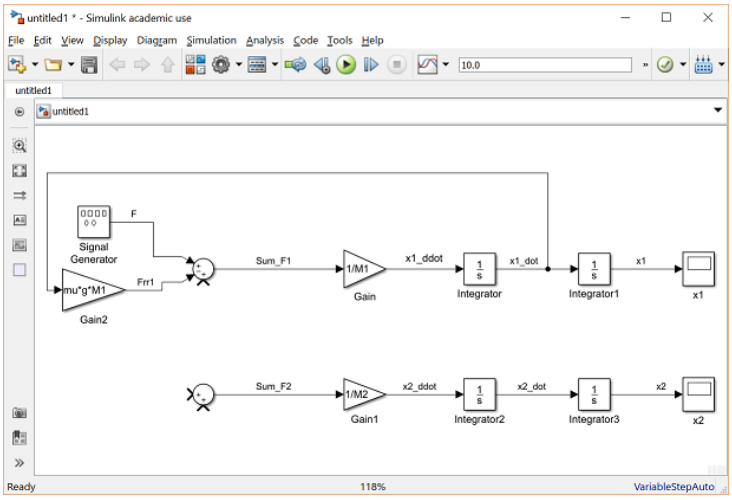

Figure 6: Modeling the friction force, extracted from [1].

Finally, the third force action on mass 1 is the elastic force between mass 1 and mass 2: $F_s = k(x_1 - x_2)$. This can be implemented in two steps: i) add a new Sum block to calculate $(x_1 - x_2)$, change the 'List of sign' field to $|-+$ rotate the block by clicking **Flip** and then **Format** (see the Figure below). Rename the output of the Sum block to $x1 - x2$. Connect x_1 and x_2 paying attention to whether they should be added or subtracted.

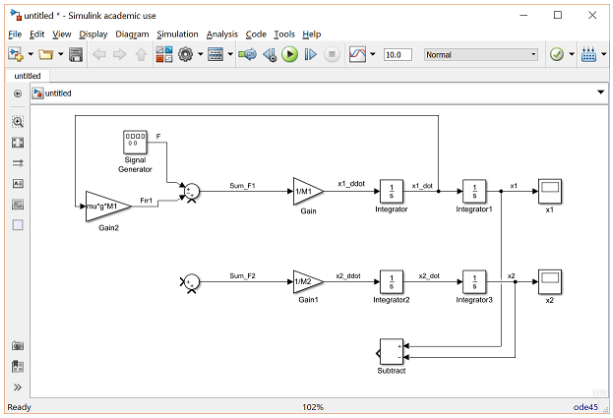

Figure 7: Modeling the elastic force, extracted from [1].

ii) Finally, the force F_s is created by connecting the output of the Sum block to a Gain block with the value of k. Then connect this force to the Sum_1 block.

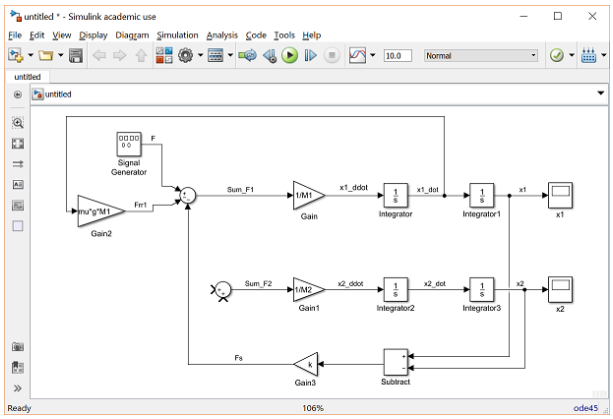

Figure 8: Dynamic equations of mass 1, extracted from [1].

Now the only missing part is the dynamic equation for mass 2, which is almost the same as for mass 1, except there is no external force acting in mass 2, and the elastic force has a negative sign. Try to implement these forces by yourself, in the end, you should have a system that looks similar to the Figure below. Now both dynamic equations that define how the system varies over time are implemented. Remember that the goal of this exercise is to design a controller that regulates the train's speed. To make this task easier, we will also define the inputs and outputs of the process (observe that almost every block in Simulink has its own input and output, but here we are using input in the sense of the signal used to control the system, and output in the sense of the signal that is measured and controlled during the simulation). In this case, the input signal is the external force F (Signal Generator block) and the output is mass 1's velocity $x1$. Add a new Scope block, connect $x1\_dot$ to it and rename the Scope to $x1\_dot$.

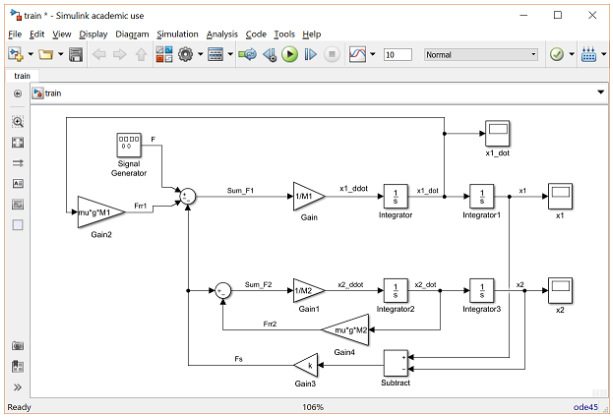

Figure 9: Dynamic model of a train implemented in Simulink, extracted from [1].

3.3 Defining variables via MATLAB

During the modeling stage, we did not hard code any parameter in the model (masses, drag constant, elastic constant, gravity, external force). This is usually good practice because it is much faster to redefine parameter variables from Matlab (just re-run a script) compared to in Simulink (double-click every block and change the parameter variable manually). Create a new m-file in MATLAB and write the following text:

$$
\begin{aligned}
M1 &= 1 \text{ kg};\\
M2 &= 0.5 \text{ kg};\\
k &= 1 \text{ N/sec};\\
F &= 1 \text{ N};\\
mu &= 0.02 \text{ sec/m};\\
g &= 9.8 \text{ m/s}^2;
\end{aligned}
$$

When you rum the script, MATLAB will create these variables, and when you run your simulation from Simulink, Simulink will read the values of k, $M1$, etc from MATLAB's workspace. Observe that you have defined the amplitude of the external force F, but you can also define it's shape and frequency. Double-click on the Signal Generator block, choose Square in the 'Wave form' field set the frequency to $0.001\ Hz$ and the amplitude to $-F$.

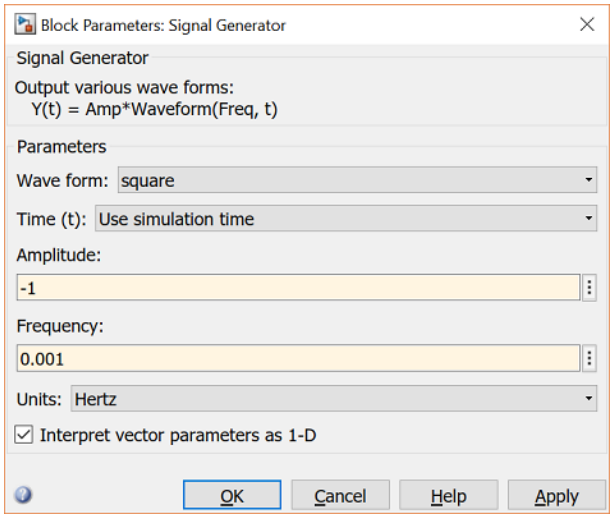

Figure 10: Signal Generator settings, extracted from [1].

Before starting a simulation, define a simulation time. In order to do that, go to **Model Configuration Parameters** in the **Simulation** menu and enter 1000 seconds as **Stop Time**. Open the Scope $x1\_dot$, execute the script to define the simulation parameters and run the simulation (green Play button). If the dynamic equations and settings have been implemented correctly, since the external force is a step shaped signal with negative and positive values, the train should go forward and backward in a frequency similar to the one used in the Signal Generator block. Compare your result to the Figure below.

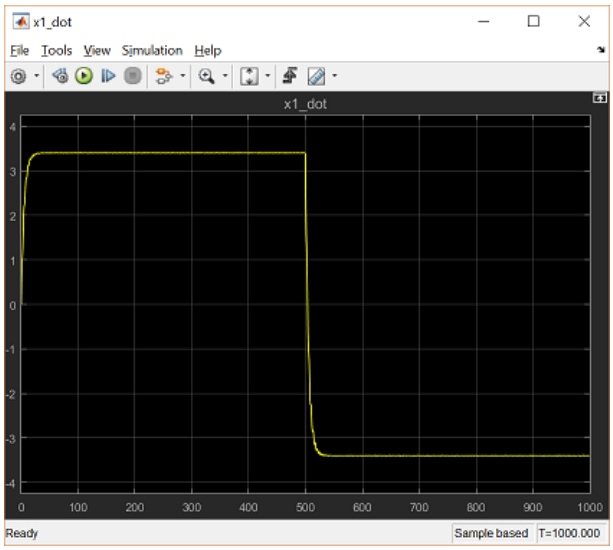

Figure 11: Velocity of mass $M1$ over time, extracted from [1].

The phase portrait is used in automatic control to analyze control systems. It consists is a plot of $\dot{x}_1$ in the y-axis and x_1 in the x-axis. In order to add a phase portrait to your model, drag a XY Graph block (in Sinks) and connect the inputs accordingly (pay attention to whether $x1_dot$ and x_1 are in the x- or y-axis). Your phase portrait should look like the one in the Figure below.

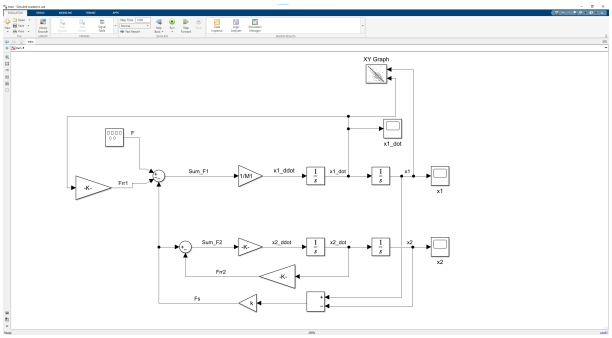

Figure 12: Final model in Simulink, extracted from [1].

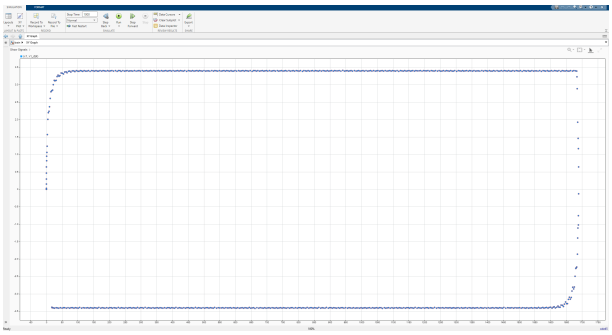

Figure 13: Phase portrait, $\dot{x}_1$ vs x_1, extracted from [1].

4 Evaluation

4.1 Oral session

The Oral session is an opportunity for you and your group to discuss the lab instructions with the lab assistants, and also an opportunity for us to make sure you have prepared to the lab. Check the Canvas room for information on how to book your session. The goal is not to evaluate your performance in the lab (which is done in a report), but rather to check that you know what you are going to do, how it is relevant to the course's theory and what is expected from you. In other words, this is a formative evaluation activity that should be more useful to you than to us.

4.2 Report

Write a report that briefly describes what you have done during the lab. Include figures that show how your final model in Simulink, as well as your phase portrait. Discuss your results, what you have learned and how you think they will be/are relevant to the course. The report grading is based on the following grading matrix:

Section Points		
4		Report Format
	2	Neat and Organized
	2	Format of tables and figures is correct
4		Completeness of Report Components
5		Introduction & Preparation(aim, objectives, list of apparatuses, references)
12		Simulink Diagram Properness
	4	Correct diagrams
	4	Completeness of labelling
	4	Output display
10		Saving Data
	5	Correctness
	5	Choice of variables
20		Graph Plotting Elaboration
	5	Correctness of plotted data
	5	Completeness of properties
	5	Proportion of display (thickness, font size, etc.)
	5	Clarity
40		Results & Analysis
	10	Correctness
	10	Technical depth
	10	Quantitative analysis
	10	Presenting challenges, ideas for improvement, applications
5		Conclusion
	5	Relevance to the objectives
		Total Possible Points: 100

The report should be submitted in Canvas before the deadline.
Grade scale:
< 80 - U, ≥ 80 - G.

See the Canvas room for more information.

5 Review history

When	Who	What
2025-04-08	DSL	Minor editing and correction.
2024-06-10	ASY	Document translated to English, minor improvements.
2024-03-19	JLI	First version created.

References

[1] *Introduction: Simulink Modeling*. URL: https://ctms.engin.umich.edu/CTMS/index.php?example=Introduction&section=SimulinkModeling.